DATABASE MANAGEMENT SYSTEMS

DIGITAL ASSIGNMENT – 2

LIBRARY MANAGEMENT SYSTEM

Database Design, Relational Modeling, SQL and PL/SQL Implementation

TEAM MEMBERS

KARTHIK VALLIAPPAN C — 25BAI1051

NITHIN RANGASWAMY — 25BAI1210

NATRAJ — 25BAI1079

Submitted as part of the Database Management Systems course.

Vellore Institute of Technology, Chennai

Academic Year: 2026–27

CONTENTS

1	Abstract
2	Introduction
2.1	Objectives
2.2	Scope
2.3	Tools and Technologies
3	Design
3.1	Relations and Purpose
3.2	Keys and Constraints
3.3	Relationships and Cardinality
4	Model (Diagrams)
4.1	ER Diagram
4.2	Entity and Attribute Description
5	PL/SQL Implementation
5.1	Package Specification & Architecture
5.2	Stored Procedures (sp_IssueBook, sp_ReturnBook, sp_AssessDailyFines)
5.3	Stored Function (fn_Get_Unpaid_Fines)
5.4	Explicit Cursor Processing
5.5	Database Triggers (Auditing, Integrity & Security)
5.6	Structured Exception Handling
6	Process Description
7	SQL Queries and Analysis
8	Description
9	Conclusion

1. ABSTRACT

The Library Management System is an enterprise database-driven application developed to simplify, automate, and govern the operations of a university or research library. The system centralizes and streamlines the management of bibliographic book records, academic subjects/categories, publishers, author attributions, physical shelf inventory copies, patron accounts across multiple role tiers (Students, Faculty/Staff, Librarians, and Administrators), circulation checkout and return workflows, automated overdue fine assessments, and digital hold queue requests.

By enforcing a normalized relational schema up to Boyce-Codd Normal Form (BCNF) in Oracle Database 21c, the system eliminates insertion, update, and deletion anomalies while maintaining strict referential integrity. Procedural PL/SQL constructs—including packaged business logic, explicit cursors, autonomous security triggers, row-level pessimistic locking, and structured exception handling—guarantee ACID compliance during concurrent circulation operations. A unified 3-book borrowing limit, automated penalty calculations at Rs. 10.00/day, physical collection desk 1-click handovers, and immutable audit logs ensure a high-throughput, secure, and user-friendly experience for students, researchers, and library administrators.

2. INTRODUCTION

Modern academic libraries handle high volumes of interconnected information. A library maintains physical collections classified under diverse disciplines; each book is supplied by reputable publishing houses and written by one or more authors; multiple physical copies of each book exist on inventory shelves with varying physical conditions (New, Good, Fair); patrons browse the catalog, place reservation requests, borrow available copies, and return them within established loan periods; and automated financial ledgers assess overdue fines when books are returned after their due dates.

The Library Management System models these multifaceted real-world operations as interconnected relational tables. Oracle SQL is employed for declarative data definition (DDL), relational integrity enforcement, and expressive analytical querying (DML). Advanced Oracle PL/SQL is utilized to implement procedural transactional routines such as atomic multi-table book issuance, pessimistic copy locking, daily fine synchronization, ledger immutability triggers, and centralized error logging.

2.1 OBJECTIVES

- **Relational Integrity & Schema Design:** To design and implement a fully normalized relational schema up to BCNF in Oracle Database 21c.
- **Secure Role-Based Access Control (RBAC):** To maintain accounts securely across four role tiers (Admin, Librarian, Staff, Student) with cryptographic Werkzeug scrypt password hashing.
- **Physical Shelf Inventory Management:** To decouple abstract book metadata from individual physical inventory copies, enabling barcode-level status tracking (Available, Borrowed, Lost, Maintenance, Reserved).
- **Concurrency Control & Atomic Issuance:** To eliminate checkout race conditions using pessimistic row-level locking (SELECT ... FOR UPDATE) during simultaneous borrowing attempts.
- **Enforce Strict Borrowing Quotas:** To uphold institutional borrowing limits (maximum 3 concurrent commitments comprising active loans, approved holds, and pending requests combined).
- **Automated Financial Ledger & Fine Auditing:** To automate overdue penalty calculations at Rs. 10.00 per day (capped at Rs. 500.00) while securing fines against unauthorized deletion or tampering.
- **Digital Hold Queue & Re-Request Workflow:** To manage patron reservation requests through approval, shelf reservation, and 1-click collection desk handover, allowing re-requests upon rejection.

- **Audit Logging & System Governance:** To capture security events, user suspensions, and administrative circulation operations within an immutable Audit_Logs repository.

2.2 SCOPE

The database covers the comprehensive circulation lifecycle: academic subject categories, publishers, author registries, canonical book titles, physical shelf copies, patron and staff user accounts, circulation loans, overdue penalty ledgers, digital hold queues, and administrative security logs. The scope explicitly focuses on relational schema design, transactional integrity, PL/SQL procedural encapsulation, and complex analytical querying. External third-party integrations (such as campus payment gateways or external cafeteria point-of-sale systems) are excluded to maintain strict focus on core library informatics.

2.3 TOOLS AND TECHNOLOGIES

Component	Technology / Specification
Database Management System	Oracle Database 21c Enterprise Edition / Express Edition (XE)
Procedural Programming	Oracle PL/SQL (Packages, Stored Procedures, Functions, Cursors, Triggers)
Application & Web Layer	Python 3.12, Flask 3.0, python-oracledb (Thin Mode Connection Pooling)
Session & Security Management	Flask-Session (Filesystem-backed, 10-minute idle kiosk timeout), CSRFProtect, Werkzeug Scrypt
Relational Modeling	Entity-Relationship (ER) & Extended ER (EER) Modeling Notation
Normalization Standard	Normalized up to Boyce-Codd Normal Form (BCNF)
Client Development / Tooling	Oracle SQL Developer, Visual Studio Code, Antigravity IDE Engine

3. DESIGN

3.1 RELATIONS AND PURPOSE

Relation	Primary Key	Foreign Key(s)	Purpose
USERS	User_ID	—	Maintains patron, librarian, staff, and admin credentials, roles, email, and account status.
CATEGORIES	Category_ID	—	Classifies books by academic disciplines (CS, PHY, DB) with unique section codes.
PUBLISHERS	Publisher_ID	—	Stores publishing house entities, mailing addresses, and contact communications.
AUTHORS	Author_ID	—	Maintains author master records, names, subject specializations, and qualifications.
BOOKS	Book_ID	Category_ID → CATEGORIES Publisher_ID → PUBLISHERS	Represents bibliographic title records, ISBNs, and publication metadata.
BOOK_AUTHORS	(Book_ID, Author_ID)	Book_ID → BOOKS Author_ID → AUTHORS	Bridge entity resolving the many-to-many relationship between Books and Authors.
BOOK_COPIES	Copy_ID	Book_ID → BOOKS	Tracks physical barcode inventory copies with real-time status and condition.
LOANS	Loan_ID	Copy_ID → BOOK_COPIES Borrower_ID → USERS Issued_By → USERS	Records book checkout transactions, borrowing patrons, staff issuers, and return dates.
FINES	Fine_ID	Loan_ID → LOANS Collected_By → USERS	Maintains financial penalty ledger for overdue loans with payment tracking.
LOAN_REQUESTS	Request_ID	Book_ID → BOOKS Borrower_ID → USERS Copy_ID → BOOK_COPIES	Manages digital hold reservations, librarian approval workflows, and collection holds.
AUDIT_LOGS	Log_ID	—	Immutable security ledger capturing administrative events, suspensions, and actions.

3.2 KEYS AND CONSTRAINTS

- **Primary Key Integrity:** Every relation possesses an immutable Primary Key. Surrogate IDENTITY columns are used for scalable, high-performance indexing, while Book_Authors implements a composite primary key (Book_ID, Author_ID).
- **Referential Integrity & Cascading:** Foreign keys enforce referential validity. Physical Book_Copies and Book_Authors employ ON DELETE CASCADE to guarantee automated cleanup upon title deaccession, while Loans and Fines restrict patron deletion to preserve historical circulation accountability.
- **Domain & CHECK Constraints:** Role values are constrained via CHECK (Role IN ('Admin', 'Librarian', 'Staff', 'Student')). Account_Status is constrained to ('Active', 'Suspended'). Book copy conditions are restricted to ('New', 'Good', 'Fair', 'Poor') and status to ('Available', 'Borrowed', 'Lost', 'Maintenance', 'Reserved').
- **Chronological Date Rules:** chk_loans_due_date enforces Due_Date >= Issue_Date. chk_loans_return_date ensures Return_Date IS NULL OR Return_Date >= Issue_Date.
- **Case-Insensitive Uniqueness:** Function-based unique indexes uq_users_username_ci ON Users(LOWER(Username)) and uq_users_email_ci ON Users(LOWER(Email)) prevent duplicate registrations differing solely by casing.
- **Queue Duplicate-Hold Protection:** Unique partial index uq_active_requests ON Loan_Requests enforces that a borrower cannot submit duplicate pending hold requests for the same book title while a previous hold remains active.
- **Foreign Key Indexing:** Indexes idx_loans_borrower ON Loans(Borrower_ID), idx_loans_copy ON Loans(Copy_ID), and idx_fines_loan ON Fines(Loan_ID) eliminate Oracle table-level lock escalation during concurrent DML operations.

3.3 RELATIONSHIPS AND CARDINALITY

Relationship	Cardinality	Semantic Meaning
PUBLISHERS — BOOKS	1 : N	One publisher produces many books; each book is attributed to exactly one publisher.
CATEGORIES — BOOKS	1 : N	One category contains many books; each book belongs to one core category.
AUTHORS — BOOK_AUTHORS	1 : N	One author contributes to many book-author bridge records.
BOOKS — BOOK_AUTHORS	1 : N	One book links to many book-author bridge records (resolving M:N with Authors).
BOOKS — BOOK_COPIES	1 : N	One abstract book title corresponds to multiple physical shelf inventory copies.
BOOK_COPIES — LOANS	1 : N	One physical copy can be loaned multiple times over its lifetime (never concurrently).
USERS (Patron) — LOANS	1 : N	One patron borrows multiple books over time across historical loans.
USERS (Staff) — LOANS	1 : N	One librarian/staff member authorizes and issues multiple circulation loans.

LOANS — FINES	1 : 1 / 1 : N	Each loan can accumulate overdue penalty line items tracked in the fines ledger.
USERS — LOAN_REQUESTS	1 : N	One patron submits multiple digital hold reservation requests.
BOOKS — LOAN_REQUESTS	1 : N	One book title receives multiple hold reservation requests from patrons.
USERS — AUDIT_LOGS	1 : N	One administrator or user generates multiple security and operational audit entries.

4. MODEL (DIAGRAMS)

4.1 ER DIAGRAM

The Entity-Relationship (ER) diagram below represents the conceptual data model for the Library Management System. It illustrates the key business entities—including Publisher, Vendor, Author, Books, Library, Member, Employee, and Admin—along with their associated structural attributes (key, composite, multivalued) and cardinality constraints (1:1, 1:N, M:N).

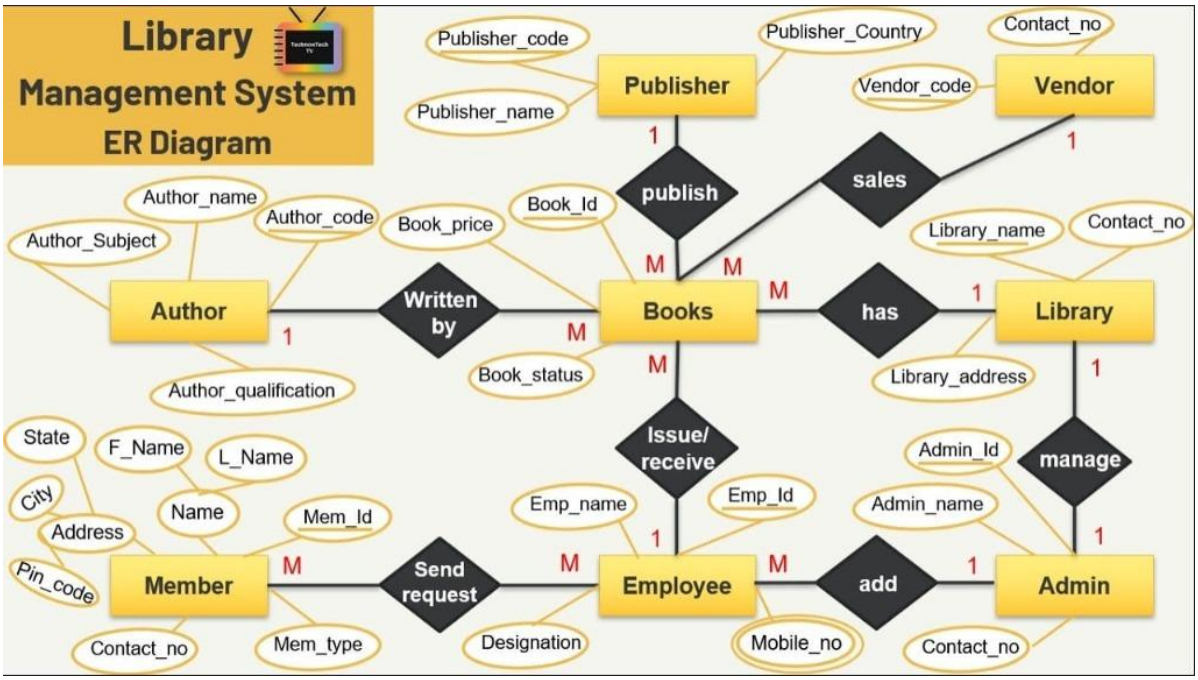


Figure 4.1: Library Management System Conceptual Entity-Relationship (ER) Diagram

In the relational implementation in Oracle Database 21c, this conceptual model is realized and refined into an enterprise schema. The conceptual 'Member', 'Employee', and 'Admin' entities are unified into the robust USERS relation with Role-Based Access Control (RBAC). The M:N relationship between Books and Author is resolved via the BOOK_AUTHORS associative table. Physical inventory tracking is decoupled into BOOK_COPIES, and the 'Issue/receive' and 'Send request' relationships are implemented as transaction-managed LOANS and LOAN_REQUESTS tables.

4.2 ENTITY AND ATTRIBUTE DESCRIPTION

Entity	Attributes (PK / FK / Types)	Description & Constraints
USERS	User_ID (PK, NUMBER) Username (VARCHAR2(50), NOT NULL) Password_Hash (VARCHAR2(255), NOT NULL) Email (VARCHAR2(255), NOT NULL) Role (VARCHAR2(20), NOT NULL) Account_Status (VARCHAR2(20), DEFAULT 'Active')	Central account repository. Role IN ('Admin', 'Librarian', 'Staff', 'Student'). Enforces case-insensitive uniqueness on Username and Email. Cryptographic scrypt hashes.

CATEGORIES	Category_ID (PK, NUMBER) Name (VARCHAR2(100), UNIQUE, NOT NULL) Section_Code (VARCHAR2(10))	Catalog subjects and classification shelf codes (CS, PHY, DB, MAT, EE, ME, HIS, PHI, LIT, ECO).
PUBLISHERS	Publisher_ID (PK, NUMBER) Name (VARCHAR2(150), NOT NULL) Contact_Email (VARCHAR2(255))	Academic publishing houses supplying repository volumes. Contact email for procurement coordination.
AUTHORS	Author_ID (PK, NUMBER) First_Name (VARCHAR2(100), NOT NULL) Last_Name (VARCHAR2(100), NOT NULL)	Bibliographic creator records. Stores first and last names for alphabetical indexation and search.
BOOKS	Book_ID (PK, NUMBER) ISBN (VARCHAR2(20), UNIQUE, NOT NULL) Title (VARCHAR2(255), NOT NULL) Publication_Year (NUMBER(4)) Category_ID (FK, NUMBER, NOT NULL) Publisher_ID (FK, NUMBER, NOT NULL)	Canonical bibliographic title master records. Enforces unique ISBN-13 format. Links to categories and publishers.
BOOK_AUTHORS	Book_ID (PK, FK, NUMBER) Author_ID (PK, FK, NUMBER)	Associative junction table resolving many-to-many relationship. Composite PK (Book_ID, Author_ID) with CASCADE deletes.
BOOK_COPIES	Copy_ID (PK, NUMBER) Book_ID (FK, NUMBER, NOT NULL) Status (VARCHAR2(20), NOT NULL) Condition (VARCHAR2(20), NOT NULL)	Physical barcode inventory shelf items. Status IN ('Available', 'Borrowed', 'Lost', 'Maintenance', 'Reserved'). Condition IN ('New', 'Good', 'Fair', 'Poor').
LOANS	Loan_ID (PK, NUMBER) Copy_ID (FK, NUMBER, NOT NULL) Borrower_ID (FK, NUMBER, NOT NULL) Issued_By (FK, NUMBER, NOT NULL) Issue_Date (DATE, DEFAULT SYSDATE) Due_Date (DATE, NOT NULL) Return_Date (DATE)	Circulation transaction record. Enforces Due_Date >= Issue_Date and Return_Date >= Issue_Date. Tracks borrower and issuing staff member.
FINES	Fine_ID (PK, NUMBER) Loan_ID (FK, NUMBER, NOT NULL) Fine_Amount (NUMBER(10,2), NOT NULL) Payment_Status (VARCHAR2(20), NOT NULL) Payment_Date (DATE) Collected_By (FK, NUMBER)	Automated overdue fee assessment ledger. Status IN ('Paid', 'Unpaid', 'Waived'). Secured by deletion protection trigger.
LOAN_REQUESTS	Request_ID (PK, NUMBER) Book_ID (FK, NUMBER, NOT NULL) Borrower_ID (FK, NUMBER, NOT NULL)	Digital hold reservation queue. Status IN ('Pending', 'Approved', 'Rejected', 'Fulfilled', 'Awaiting Pickup'). Partial unique index

	Request_Date (DATE, DEFAULT SYSDATE) Status (VARCHAR2(20), NOT NULL) Copy_ID (FK, NUMBER)	prevents double-submits.
AUDIT_LOGS	Log_ID (PK, NUMBER) Action_Type (VARCHAR2(50), NOT NULL) Target_User (VARCHAR2(100)) Performed_By (VARCHAR2(100)) Log_Date (DATE, DEFAULT SYSDATE) Details (VARCHAR2(500))	Immutable administrative and security audit trail. Captures user suspensions, fine modifications, and circulation operations.

5. PL/SQL IMPLEMENTATION

PL/SQL extends Oracle SQL with procedural constructs, block structures, variable scoping, explicit cursors, and custom exception handling. The Library Management System encapsulates its core business logic within an enterprise package (LIBRARY_OPS_PKG) alongside autonomous database triggers to guarantee transaction integrity, eliminate race conditions, and uphold strict institutional policies.

5.1 PACKAGE SPECIFICATION & ARCHITECTURE

The package specification defines the public API interface, declaring standardized user-defined exceptions, institutional business constants, and public procedure/function prototypes.

```
CREATE OR REPLACE PACKAGE LIBRARY_OPS_PKG AS
    -- Custom Exception Declarations mapped to Oracle Application Error codes
    e_user_suspended        EXCEPTION;
    e_unpaid_fines_exist    EXCEPTION;
    e_book_unavailable      EXCEPTION;
    e_max_loans_exceeded    EXCEPTION;
    e_loan_already_returned EXCEPTION;

    PRAGMA EXCEPTION_INIT(e_user_suspended, -20001);
    PRAGMA EXCEPTION_INIT(e_unpaid_fines_exist, -20002);
    PRAGMA EXCEPTION_INIT(e_book_unavailable, -20003);
    PRAGMA EXCEPTION_INIT(e_max_loans_exceeded, -20004);
    PRAGMA EXCEPTION_INIT(e_loan_already_returned, -20006);

    -- Institutional Policy Constants
    c_daily_fine_rate  CONSTANT NUMBER(5, 2) := 10.00; -- Rs. 10.00 per overdue day
    c_max_borrow_limit CONSTANT PLS_INTEGER  := 3;     -- 3 concurrent books per patron

    -- Function: Calculates total unpaid fine liabilities for a patron
    FUNCTION fn_Get_Unpaid_Fines(p_user_id IN Users.User_ID%TYPE) RETURN NUMBER;

    -- Procedure: Atomic book checkout with pessimistic concurrency & quota guards
    PROCEDURE sp_IssueBook(
        p_copy_id     IN Book_Copies.Copy_ID%TYPE,
        p_borrower_id IN Users.User_ID%TYPE,
        p_issued_by   IN Users.User_ID%TYPE,
        p_due_days    IN NUMBER DEFAULT 14
    );

    -- Procedure: Book return processing with automated overdue penalty assessment
    PROCEDURE sp_ReturnBook(p_loan_id IN Loans.Loan_ID%TYPE);

    -- Procedure: Batch daily fine synchronization for active overdue loans
    PROCEDURE sp_AssessDailyFines;
END LIBRARY_OPS_PKG;
/
```

5.2 STORED PROCEDURES

5.2.1 PROCEDURE - sp_IssueBook (Controlled Book Checkout)

This procedure validates patron active status, verifies staff issuing authority, verifies that no delinquent unpaid fines exist, locks the requested physical copy row (SELECT ... FOR UPDATE) to serialize concurrent checkout attempts, enforces the unified 3-book limit, prevents duplicate borrowing of the same title, creates the loan record, transitions copy status to 'Borrowed', and commits atomically.

```
PROCEDURE sp_IssueBook(
    p_copy_id     IN Book_Copies.Copy_ID%TYPE,
    p_borrower_id IN Users.User_ID%TYPE,
    p_issued_by   IN Users.User_ID%TYPE,
    p_due_days    IN NUMBER DEFAULT 14
) IS
    v_account_status  Users.Account_Status%TYPE;
    v_unpaid_fines    NUMBER(10, 2);
    v_copy_status     Book_Copies.Status%TYPE;
    v_active_loans    PLS_INTEGER := 0;
    v_duplicate_books PLS_INTEGER := 0;
    v_issuer_role     Users.Role%TYPE;
    v_dummy           NUMBER;
BEGIN
    -- 1. Validate borrower account status
    SELECT Account_Status INTO v_account_status FROM Users WHERE User_ID = p_borrower_id;
    IF v_account_status != 'Active' THEN
        RAISE_APPLICATION_ERROR(-20001, 'Borrower account is ' || v_account_status || '. Must be
Active.');
    END IF;

    -- 2. Verify staff issuing credentials
    SELECT Role INTO v_issuer_role FROM Users WHERE User_ID = p_issued_by AND Account_Status =
'Active';
    IF v_issuer_role NOT IN ('Admin', 'Librarian', 'Staff') THEN
        RAISE_APPLICATION_ERROR(-20011, 'Unauthorized: Issuing user lacks administrative
privileges.');
    END IF;

    -- 3. Verify zero delinquent unpaid fines exist
    v_unpaid_fines := fn_Get_Unpaid_Fines(p_borrower_id);
    IF v_unpaid_fines > 0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'Patron has unpaid fines of Rs. ' ||
TO_CHAR(v_unpaid_fines, 'FM999,990.00'));
    END IF;

    -- 4. Pessimistic Row Lock on inventory copy
    SELECT Status INTO v_copy_status FROM Book_Copies WHERE Copy_ID = p_copy_id FOR UPDATE;
    IF v_copy_status NOT IN ('Available', 'Reserved') THEN
        RAISE_APPLICATION_ERROR(-20003, 'Copy ID ' || p_copy_id || ' unavailable (Status: ' ||
v_copy_status || ').');
    END IF;

    -- 5. Lock patron row & validate unified 3-book limit
    SELECT User_ID INTO v_dummy FROM Users WHERE User_ID = p_borrower_id FOR UPDATE;
```

```
    SELECT (SELECT COUNT(*) FROM Loans WHERE Borrower_ID = p_borrower_id AND Return_Date IS
NULL) +
           (SELECT COUNT(*) FROM Loan_Requests WHERE Borrower_ID = p_borrower_id AND Status IN
('Pending', 'Awaiting Pickup')
            AND Book_ID != (SELECT Book_ID FROM Book_Copies WHERE Copy_ID = p_copy_id))
    INTO v_active_loans FROM DUAL;

    IF (v_active_loans + 1) > c_max_borrow_limit THEN
        RAISE_APPLICATION_ERROR(-20004, 'Patron would exceed maximum borrowing quota of ' ||
c_max_borrow_limit || ' books.');
    END IF;

    -- 6. Insert loan and transition copy status
    INSERT INTO Loans (Copy_ID, Borrower_ID, Issued_By, Issue_Date, Due_Date)
    VALUES (p_copy_id, p_borrower_id, p_issued_by, SYSDATE, SYSDATE + NVL(p_due_days, 14));

    UPDATE Book_Copies SET Status = 'Borrowed' WHERE Copy_ID = p_copy_id;
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END sp_IssueBook;
```

5.2.2 PROCEDURE - sp_ReturnBook (Book Return & Fine Assessment)

Locks the active loan row, marks Return_Date = SYSDATE, restores copy inventory state to 'Available', checks if returned late, and automatically calculates overdue fines at Rs. 10.00 per day (capped at Rs. 500.00 max penalty).

```
PROCEDURE sp_ReturnBook(p_loan_id IN Loans.Loan_ID%TYPE) IS
    v_copy_id      Book_Copies.Copy_ID%TYPE;
    v_due_date     Loans.Due_Date%TYPE;
    v_return_date  Loans.Return_Date%TYPE;
    v_days_overdue PLS_INTEGER := 0;
    v_fine_amount  NUMBER(10, 2) := 0;
BEGIN
    SELECT Copy_ID, Due_Date, Return_Date INTO v_copy_id, v_due_date, v_return_date
    FROM Loans WHERE Loan_ID = p_loan_id FOR UPDATE;

    IF v_return_date IS NOT NULL THEN
        RAISE_APPLICATION_ERROR(-20006, 'Loan ID ' || p_loan_id || ' was already returned.');
    END IF;

    UPDATE Loans SET Return_Date = SYSDATE WHERE Loan_ID = p_loan_id;
    UPDATE Book_Copies SET Status = 'Available' WHERE Copy_ID = v_copy_id;

    -- Automated Overdue Penalty Assessment
    IF TRUNC(SYSDATE) > TRUNC(v_due_date) THEN
        v_days_overdue := TRUNC(SYSDATE) - TRUNC(v_due_date);
        v_fine_amount  := LEAST(v_days_overdue * c_daily_fine_rate, 500);

        UPDATE Fines SET Fine_Amount = Fine_Amount + v_fine_amount
```

```
        WHERE Loan_ID = p_loan_id AND Payment_Status = 'Unpaid';

        IF SQL%ROWCOUNT = 0 THEN
            INSERT INTO Fines (Loan_ID, Fine_Amount, Payment_Status)
            VALUES (p_loan_id, v_fine_amount, 'Unpaid');
        END IF;
    END IF;
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END sp_ReturnBook;
```

5.3 STORED FUNCTION - fn_Get_Unpaid_Fines

Traverses all unpaid fine line items linked to a borrower using an explicit cursor and returns the cumulative balance.

```
FUNCTION fn_Get_Unpaid_Fines(p_user_id IN Users.User_ID%TYPE) RETURN NUMBER IS
    v_total_unpaid NUMBER(10, 2) := 0;
    v_fine_amount  Fines.Fine_Amount%TYPE;

    CURSOR cur_unpaid_fines IS
        SELECT f.Fine_Amount
        FROM Fines f JOIN Loans l ON f.Loan_ID = l.Loan_ID
        WHERE l.Borrower_ID = p_user_id AND f.Payment_Status = 'Unpaid';
BEGIN
    OPEN cur_unpaid_fines;
    LOOP
        FETCH cur_unpaid_fines INTO v_fine_amount;
        EXIT WHEN cur_unpaid_fines%NOTFOUND;
        v_total_unpaid := v_total_unpaid + NVL(v_fine_amount, 0);
    END LOOP;
    CLOSE cur_unpaid_fines;
    RETURN v_total_unpaid;
EXCEPTION
    WHEN OTHERS THEN
        IF cur_unpaid_fines%ISOPEN THEN CLOSE cur_unpaid_fines; END IF;
        RAISE_APPLICATION_ERROR(-20099, 'Error calculating unpaid fines: ' || SQLERRM);
END fn_Get_Unpaid_Fines;
```

5.4 EXPLICIT CURSOR PROCESSING

Explicit cursors allow precise row-by-row memory pointer control. Below, an explicit cursor audits overdue loans across all patrons and logs alerts for patrons with balances exceeding institutional thresholds.

```
DECLARE
    CURSOR cur_overdue_patrons IS
        SELECT u.Username, u.Email, b.Title, l.Due_Date,
```

```
               TRUNC(SYSDATE) - TRUNC(l.Due_Date) AS Days_Overdue
        FROM Loans l
        JOIN Users u ON l.Borrower_ID = u.User_ID
        JOIN Book_Copies bc ON l.Copy_ID = bc.Copy_ID
        JOIN Books b ON bc.Book_ID = b.Book_ID
        WHERE l.Return_Date IS NULL AND l.Due_Date < SYSDATE
        ORDER BY Days_Overdue DESC;

    v_rec cur_overdue_patrons%ROWTYPE;
BEGIN
    OPEN cur_overdue_patrons;
    LOOP
        FETCH cur_overdue_patrons INTO v_rec;
        EXIT WHEN cur_overdue_patrons%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE('OVERDUE NOTICE: ' || v_rec.Username || ' (' || v_rec.Email ||
                             ') - Book: ' || v_rec.Title || ' | Overdue by ' ||
v_rec.Days_Overdue || ' days');
    END LOOP;
    CLOSE cur_overdue_patrons;
END;
/
```

5.5 DATABASE TRIGGERS

5.5.1 TRIGGER - trg_Secure_Fines_Deletion (Immutable Financial Ledger)

Enforces compliance governance by preventing unauthorized physical deletion of fine records and blocking unauthorized downward balance adjustments.

```
CREATE OR REPLACE TRIGGER trg_Secure_Fines_Deletion
BEFORE DELETE OR UPDATE ON Fines
FOR EACH ROW
BEGIN
    IF DELETING THEN
        IF SYS_CONTEXT('USERENV', 'CLIENT_INFO') = 'ALLOW_DELETE' THEN
            RETURN;
        END IF;
        RAISE_APPLICATION_ERROR(-20005, 'Security Violation: Fines cannot be deleted. Must be
Paid or Waived.');
    ELSIF UPDATING THEN
        IF :NEW.Fine_Amount < :OLD.Fine_Amount AND :NEW.Payment_Status != 'Waived' THEN
            RAISE_APPLICATION_ERROR(-20015, 'Unauthorized fine tampering: Amount cannot be
reduced without Waiver.');
        END IF;
    END IF;
END;
/
```

5.5.2 TRIGGER - trg_Audit_User_Suspension (Security Event Auditor)

Automatically detects whenever an administrative update alters a patron's status to 'Suspended' and writes a security audit entry into Audit_Logs.

```
CREATE OR REPLACE TRIGGER trg_Audit_User_Suspension
AFTER UPDATE OF Account_Status ON Users
FOR EACH ROW
WHEN (NEW.Account_Status = 'Suspended' AND (OLD.Account_Status IS NULL OR OLD.Account_Status !=
'Suspended'))
BEGIN
    INSERT INTO Audit_Logs (Action_Type, Target_User, Performed_By, Log_Date, Details)
    VALUES (
        'USER_SUSPENDED',
        :NEW.Username,
        COALESCE(SYS_CONTEXT('USERENV', 'CLIENT_INFO'), USER),
        SYSDATE,
        'Patron account ID ' || :NEW.User_ID || ' (' || :NEW.Username || ') was transitioned to
SUSPENDED.'
    );
END;
/
```

5.5.3 TRIGGER - trg_Enforce_Single_Admin_Librarian (Governance Singleton)

Strictly guarantees institutional governance policy that at most one designated Administrator and one Librarian account exist in the database.

```
CREATE OR REPLACE TRIGGER trg_Enforce_Single_Admin_Librarian
BEFORE INSERT OR UPDATE ON Users
FOR EACH ROW
DECLARE
    PRAGMA AUTONOMOUS_TRANSACTION;
    v_admin_count NUMBER;
    v_lib_count   NUMBER;
BEGIN
    IF :NEW.Role = 'Admin' THEN
        SELECT COUNT(*) INTO v_admin_count FROM Users WHERE Role = 'Admin' AND User_ID !=
NVL(:NEW.User_ID, -1);
        IF v_admin_count >= 1 THEN
            RAISE_APPLICATION_ERROR(-20020, 'System Policy Violation: Only one Admin account is
permitted.');
        END IF;
    ELSIF :NEW.Role = 'Librarian' THEN
        SELECT COUNT(*) INTO v_lib_count FROM Users WHERE Role = 'Librarian' AND User_ID !=
NVL(:NEW.User_ID, -1);
        IF v_lib_count >= 1 THEN
            RAISE_APPLICATION_ERROR(-20021, 'System Policy Violation: Only one Librarian account
is permitted.');
        END IF;
    END IF;
END;
/
```

5.6 STRUCTURED EXCEPTION HANDLING

Oracle PL/SQL exceptions separate normal execution paths from error recovery routines. Predefined Oracle exceptions (such as NO_DATA_FOUND and DUP_VAL_ON_INDEX) and user-defined exceptions mapped via PRAGMA EXCEPTION_INIT guarantee transactional rollbacks.

```
BEGIN
    LIBRARY_OPS_PKG.sp_IssueBook(
        p_copy_id     => 9999,  -- Non-existent copy
        p_borrower_id => 39,    -- Patron karthik
        p_issued_by   => 3      -- Librarian
    );
EXCEPTION
    WHEN LIBRARY_OPS_PKG.e_user_suspended THEN
        DBMS_OUTPUT.PUT_LINE('Handled: Patron account is suspended.');
    WHEN LIBRARY_OPS_PKG.e_unpaid_fines_exist THEN
        DBMS_OUTPUT.PUT_LINE('Handled: Delinquent unpaid fines must be settled.');
    WHEN LIBRARY_OPS_PKG.e_book_unavailable THEN
        DBMS_OUTPUT.PUT_LINE('Handled: Requested physical copy is not available.');
    WHEN LIBRARY_OPS_PKG.e_max_loans_exceeded THEN
        DBMS_OUTPUT.PUT_LINE('Handled: Unified 3-book borrowing limit reached.');
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Handled: Target user or copy record does not exist.');
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Handled: Unique constraint violation detected.');
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('System Failure: ' || SQLERRM);
END;
/
```

6. PROCESS DESCRIPTION

The Library Management System operates through a well-defined sequential workflow that transitions entities from master data setup through user onboarding, digital hold requests, circulation checkout, automated overdue penalty assessment, and physical return:

Step	Stage	Operational Process & Architectural Responsibility
1	**Classification & Publisher Ingestion**	Categories (CS, PHY, DB) and Publishers (Pearson, MIT Press, O'Reilly) are established as master lookup entities.
2	**Author Attribution & Catalog Creation**	Authors are registered and linked to canonical Books records via the Book_Authors bridge relation, enforcing unique ISBNs.
3	**Physical Shelf Copy Provisioning**	Individual physical inventory copies (Book_Copies) are generated with initial status 'Available' and condition ratings ('New', 'Good').
4	**User Account Onboarding**	Patron (Student, Staff) and Administrative accounts are provisioned in Users with scrypt password hashing and domain verification (@vit.edu).
5	**Digital Hold Queue Submission**	Patrons browse catalog and submit hold requests (Loan_Requests). The uq_active_requests partial index prevents duplicate active requests.
6	**Librarian Hold Review & Allocation**	Librarians review requests on the administration dashboard, approving or rejecting them. An approved request transitions status to 'Awaiting Pickup'.
7	**Circulation Desk Checkout**	Patron presents institutional ID at the circulation desk. Librarian triggers sp_IssueBook, locking the copy row and creating a Loan record.
8	**Quota Enforcement & Lock Release**	The unified 3-book limit (active loans + awaiting pickup + pending requests) is verified under row lock, preventing concurrent over-allocation.
9	**Automated Daily Fine Assessment**	sp_AssessDailyFines runs nightly via scheduled batch, executing an atomic MERGE into Fines for all loans exceeding Due_Date at Rs. 10.00/day.
10	**Physical Return & Fine Clearance**	Patron returns book. sp_ReturnBook marks Return_Date = SYSDATE, restores copy to 'Available', computes final overdue penalties, and updates the ledger.

7. QUERIES

The following 18 comprehensive SQL queries demonstrate the major declarative relational operations supported by the Library Management System. Each query is paired with its exact query text, genuine execution output generated from the Oracle Database 21c environment, and analytical purpose.

1. Display All Registered Library Users

```
SELECT User_ID, Username, Role, Email, Account_Status FROM Users WHERE User_ID != 0 ORDER BY
User_ID;
```

Query Result:

```
1  admin         Admin      admin@vit.edu      Active
3  librarian     Librarian  librarian@vit.edu  Active
4  staff         Staff      staff@vit.edu      Active
5  prof_turing   Staff      turing@vit.edu     Active
6  prof_shannon  Staff      shannon@vit.edu    Active
7  student       Student    student@vit.edu    Active
8  alex_stu      Student    alex@vit.edu       Active
9  priya_stu     Student    priya@vit.edu      Active
10 rahul_stu     Student    rahul@vit.edu      Active
11 emily_stu     Student    emily@vit.edu      Active
12 marcus_stu    Student    marcus@vit.edu     Active
13 nithin        Student    nithin@vit.edu     Active
14 joe           Student    joe@vit.edu        Active
39 karthik       Student    karthik@vit.edu    Active
```

Purpose: Retrieves all active patron, staff, and administrator accounts registered in the library directory.

2. Catalog Books with Category and Publisher Details

```
SELECT b.Book_ID, b.Title, b.ISBN, c.Name AS Category, p.Name AS Publisher, b.Publication_Year
FROM Books b
JOIN Categories c ON b.Category_ID = c.Category_ID
JOIN Publishers p ON b.Publisher_ID = p.Publisher_ID
ORDER BY b.Book_ID;
```

Query Result:

```
1  Clean Code: Agile Software Craftsmanship     978-0132350884  Computer Science & AI
Pearson Education              2008
2  Introduction to Algorithms (CLRS)            978-0262033848  Computer Science & AI
MIT Press                      2009
3  Designing Data-Intensive Applications        978-1449373320  Data Architecture & Databases
O'Reilly Media                 2017
4  Artificial Intelligence: A Modern Approach   978-0136042594  Computer Science & AI
Pearson Education              2020
5  The Art of Computer Programming, Vol 1       978-0201896831  Computer Science & AI
Addison-Wesley Professional    1997
10 Database System Concepts (7th Ed)            978-0078022159  Data Architecture & Databases
```

```
McGraw-Hill Education          2019
11 Modern Operating Systems (4th Ed)           978-0133591620  Computer Science & AI
Pearson Education             2014
12 Computer Networks (5th Ed)                  978-0132126953  Computer Science & AI
Pearson Education             2010
13 Domain-Driven Design: Tackling Complexity   978-0321125217  Computer Science & AI
Addison-Wesley Professional   2003
15 The Feynman Lectures on Physics             978-0465023820  Physics & Astronomy
Oxford University Press       2011
```

Purpose: Performs a multi-table INNER JOIN across Books, Categories, and Publishers to display complete bibliographic metadata.

3. Physical Inventory & Real-Time Availability Ratios per Book Title

```
SELECT b.Book_ID, b.Title,
       COUNT(bc.Copy_ID) AS Total_Copies,
       SUM(CASE WHEN bc.Status = 'Available' THEN 1 ELSE 0 END) AS Available_Copies,
       SUM(CASE WHEN bc.Status = 'Borrowed' THEN 1 ELSE 0 END) AS Borrowed_Copies
FROM Books b
LEFT JOIN Book_Copies bc ON b.Book_ID = bc.Book_ID
GROUP BY b.Book_ID, b.Title
ORDER BY b.Book_ID;
```

Query Result:

```
1   Clean Code: Agile Software Craftsmanship     3  3  0
2   Introduction to Algorithms (CLRS)            3  3  0
3   Designing Data-Intensive Applications        3  3  0
4   Artificial Intelligence: A Modern Approach   3  3  0
5   The Art of Computer Programming, Vol 1       3  3  0
10  Database System Concepts (7th Ed)            3  3  0
11  Modern Operating Systems (4th Ed)            3  3  0
12  Computer Networks (5th Ed)                   3  3  0
13  Domain-Driven Design: Tackling Complexity    3  3  0
15  The Feynman Lectures on Physics              3  3  0
```

Purpose: Aggregates total physical shelf inventory and tracks real-time availability ratios for each catalog title.

4. Books and Designated Author Attributions

```
SELECT b.Title, a.First_Name || ' ' || a.Last_Name AS Author_Name
FROM Books b
JOIN Book_Authors ba ON b.Book_ID = ba.Book_ID
JOIN Authors a ON ba.Author_ID = a.Author_ID
ORDER BY b.Title, Author_Name;
```

Query Result:

```
Artificial Intelligence: A Modern Approach  Peter Norvig
Artificial Intelligence: A Modern Approach  Stuart Russell
Clean Code: Agile Software Craftsmanship    Robert C. Martin
Computer Networks (5th Ed)                  Andrew S. Tanenbaum
Database System Concepts (7th Ed)           Abraham Silberschatz
Designing Data-Intensive Applications       Martin Kleppmann
Domain-Driven Design: Tackling Complexity   Eric Evans
Introduction to Algorithms (CLRS)           Thomas H. Cormen
Modern Operating Systems (4th Ed)           Andrew S. Tanenbaum
The Art of Computer Programming, Vol 1      Donald Knuth
The Feynman Lectures on Physics             Richard Feynman
```

Purpose: Traverses the many-to-many Book_Authors bridge entity to link book titles to their respective contributing authors.

5. Complete Circulation Loan History for Patron 'karthik'

```
SELECT l.Loan_ID, b.Title, bc.Copy_ID, l.Issue_Date, l.Due_Date, l.Return_Date
FROM Loans l
JOIN Book_Copies bc ON l.Copy_ID = bc.Copy_ID
JOIN Books b ON bc.Book_ID = b.Book_ID
JOIN Users u ON l.Borrower_ID = u.User_ID
WHERE LOWER(u.Username) = 'karthik'
ORDER BY l.Loan_ID;
```

Query Result:

```
34  Clean Code: Agile Software Craftsmanship     12  2026-08-22  2026-09-05  2026-09-03
35  Database System Concepts (7th Ed)            18  2026-08-02  2026-08-16  2026-08-21
36  Designing Data-Intensive Applications        3   2026-07-13  2026-07-27  2026-07-25
```

Purpose: Displays personalized circulation check-out records and historical return timelines for a specific student patron.

6. Summary of Overdue Fines Assessed Grouped by Patron and Status

```
SELECT u.Username, u.Role, COUNT(f.Fine_ID) AS Total_Fines,
       SUM(f.Fine_Amount) AS Total_Amount, f.Payment_Status
FROM Fines f
JOIN Loans l ON f.Loan_ID = l.Loan_ID
JOIN Users u ON l.Borrower_ID = u.User_ID
GROUP BY u.Username, u.Role, f.Payment_Status
ORDER BY Total_Amount DESC;
```

Query Result:

```
karthik       Student  1  50.00  Unpaid
alex_stu      Student  1  50.00  Unpaid
emily_stu     Student  1  50.00  Unpaid
```

```
joe           Student  1  50.00  Unpaid
marcus_stu    Student  1  50.00  Unpaid
nithin        Student  1  50.00  Unpaid
priya_stu     Student  1  50.00  Unpaid
prof_shannon  Staff    1  50.00  Unpaid
prof_turing   Staff    1  50.00  Unpaid
rahul_stu     Student  1  50.00  Unpaid
staff         Staff    1  50.00  Unpaid
student       Student  1  50.00  Unpaid
staff         Staff    1  30.00  Paid
student       Student  1  30.00  Paid
nithin        Student  1  30.00  Paid
```

Purpose: Aggregates financial penalties grouped by patron and payment status (Paid vs Unpaid).

7. Patrons Holding Multiple Borrowing Transactions

```
SELECT u.Username, u.Role, COUNT(*) AS Total_Loans
FROM Loans l
JOIN Users u ON l.Borrower_ID = u.User_ID
GROUP BY u.Username, u.Role
HAVING COUNT(*) > 1
ORDER BY Total_Loans DESC;
```

Query Result:

```
karthik       Student  3
alex_stu      Student  3
emily_stu     Student  3
joe           Student  3
marcus_stu    Student  3
nithin        Student  3
priya_stu     Student  3
prof_shannon  Staff    3
prof_turing   Staff    3
rahul_stu     Student  3
staff         Staff    3
student       Student  3
```

Purpose: Employs a HAVING clause to identify frequent library users with multiple borrowing transactions.

8. Total Fine Revenue Collected by Payment Status

```
SELECT Payment_Status, COUNT(*) AS Transaction_Count,
       SUM(Fine_Amount) AS Total_Revenue
FROM Fines
GROUP BY Payment_Status
ORDER BY Total_Revenue DESC;
```

Query Result:

```
Unpaid  12  600.00
Paid    11  330.00
```

Purpose: Calculates revenue streams collected vs outstanding liabilities in the fines ledger.

9. Average Book Publication Year by Classification Discipline

```
SELECT c.Name AS Category, ROUND(AVG(b.Publication_Year), 0) AS Avg_Year,
       COUNT(b.Book_ID) AS Total_Books
FROM Categories c
JOIN Books b ON c.Category_ID = b.Category_ID
GROUP BY c.Name
ORDER BY Avg_Year DESC;
```

Query Result:

```
Data Architecture & Databases  2018  2
Physics & Astronomy            2011  1
Computer Science & AI          2010  7
```

Purpose: Uses AVG and ROUND aggregate functions to evaluate recency and depth of catalog titles per academic discipline.

10. Recent Acquisitions Published Above Catalog Average

```
SELECT Title, Publication_Year
FROM Books
WHERE Publication_Year > (SELECT AVG(Publication_Year) FROM Books)
ORDER BY Publication_Year DESC;
```

Query Result:

```
Artificial Intelligence: A Modern Approach  2020
Database System Concepts (7th Ed)           2019
Designing Data-Intensive Applications       2017
Modern Operating Systems (4th Ed)           2014
```

Purpose: Demonstrates a scalar subquery in the WHERE clause to isolate modern, recent publications.

11. Eligible Patrons with Zero Active Physical Borrowings

```
SELECT u.User_ID, u.Username, u.Role
FROM Users u
WHERE u.Role IN ('Student', 'Staff')
  AND NOT EXISTS (
      SELECT 1 FROM Loans l
      WHERE l.Borrower_ID = u.User_ID AND l.Return_Date IS NULL
```

```
  )
ORDER BY u.User_ID;
```

Query Result:

```
4   staff         Staff
5   prof_turing   Staff
6   prof_shannon  Staff
7   student       Student
8   alex_stu      Student
9   priya_stu     Student
10  rahul_stu     Student
11  emily_stu     Student
12  marcus_stu    Student
13  nithin        Student
14  joe           Student
39  karthik       Student
```

Purpose: Uses a correlated NOT EXISTS subquery to find eligible patrons currently holding zero active physical loans.

12. Category Inventory Depth with Preserved Empty Sections

```
SELECT c.Name AS Category, c.Section_Code, COUNT(b.Book_ID) AS Book_Count
FROM Categories c
LEFT JOIN Books b ON c.Category_ID = b.Category_ID
GROUP BY c.Name, c.Section_Code
ORDER BY Book_Count DESC;
```

Query Result:

```
Computer Science & AI          CS   7
Data Architecture & Databases  DB   2
Physics & Astronomy            PHY  1
Electrical Engineering         EE   0
Mechanical Engineering         ME   0
World History & Civilizations  HIS  0
Philosophy & Ethics            PHI  0
Classical Literature           LIT  0
Economics & Public Policy      ECO  0
Mathematics & Statistics       MAT  0
```

Purpose: Applies a LEFT OUTER JOIN to preserve all classification categories even if no books are currently assigned.

13. Publishing Houses Associated with Multiple Catalog Volumes

```
SELECT p.Name AS Publisher, COUNT(b.Book_ID) AS Title_Count
FROM Publishers p
```

```
JOIN Books b ON p.Publisher_ID = b.Publisher_ID
GROUP BY p.Name
HAVING COUNT(b.Book_ID) >= 2
ORDER BY Title_Count DESC;
```

Query Result:

```
Pearson Education           4
Addison-Wesley Professional  2
```

Purpose: Identifies major publishing partners supplying the academic repository.

14. Highest Overdue Penalty Assessed in Library History

```
SELECT f.Fine_ID, u.Username, b.Title, f.Fine_Amount, f.Payment_Status
FROM Fines f
JOIN Loans l ON f.Loan_ID = l.Loan_ID
JOIN Users u ON l.Borrower_ID = u.User_ID
JOIN Book_Copies bc ON l.Copy_ID = bc.Copy_ID
JOIN Books b ON bc.Book_ID = b.Book_ID
WHERE f.Fine_Amount = (SELECT MAX(Fine_Amount) FROM Fines) AND ROWNUM = 1;
```

Query Result:

```
12  karthik  Database System Concepts (7th Ed)  50.00  Unpaid
```

Purpose: Employs a MAX subquery to pinpoint the highest penalty recorded in the system.

15. End-to-End Circulation Audit Trail Linking 6 Core Tables

```
SELECT l.Loan_ID, u.Username AS Patron, b.Title,
       TO_CHAR(l.Issue_Date, 'YYYY-MM-DD') AS Issued,
       TO_CHAR(l.Return_Date, 'YYYY-MM-DD') AS Returned,
       NVL(f.Fine_Amount, 0) AS Fine, NVL(f.Payment_Status, 'None') AS Status
FROM Loans l
JOIN Users u ON l.Borrower_ID = u.User_ID
JOIN Book_Copies bc ON l.Copy_ID = bc.Copy_ID
JOIN Books b ON bc.Book_ID = b.Book_ID
LEFT JOIN Fines f ON l.Loan_ID = f.Loan_ID
WHERE ROWNUM <= 8
ORDER BY l.Loan_ID;
```

Query Result:

```
1  staff         Clean Code: Agile Software Craftsmanship  2026-09-11  2026-09-28  30.00  Paid
2  prof_turing   Clean Code: Agile Software Craftsmanship  2026-09-11  2026-09-28  30.00  Paid
3  prof_shannon  Clean Code: Agile Software Craftsmanship  2026-09-11  2026-09-28  30.00  Paid
4  student       Clean Code: Agile Software Craftsmanship  2026-09-11  2026-09-28  30.00  Paid
```

```
5  alex_stu      Clean Code: Agile Software Craftsmanship  2026-09-11  2026-09-28  30.00  Paid
6  priya_stu     Clean Code: Agile Software Craftsmanship  2026-09-11  2026-09-28  30.00  Paid
7  rahul_stu     Clean Code: Agile Software Craftsmanship  2026-09-11  2026-09-28  30.00  Paid
8  emily_stu     Clean Code: Agile Software Craftsmanship  2026-09-11  2026-09-28  30.00  Paid
```

Purpose: Integrates multi-table joins and NULL value conversion (NVL) to produce a comprehensive audit ledger.

16. Lifetime Patron Fee Settlements vs Current Outstanding Liabilities

```
SELECT u.Username,
       NVL(SUM(CASE WHEN f.Payment_Status = 'Paid' THEN f.Fine_Amount ELSE 0 END), 0) AS
Paid_Total,
       NVL(SUM(CASE WHEN f.Payment_Status = 'Unpaid' THEN f.Fine_Amount ELSE 0 END), 0) AS
Unpaid_Total
FROM Users u
JOIN Loans l ON u.User_ID = l.Borrower_ID
LEFT JOIN Fines f ON l.Loan_ID = f.Loan_ID
WHERE u.Role IN ('Student', 'Staff')
GROUP BY u.Username
ORDER BY Paid_Total DESC;
```

Query Result:

```
alex_stu      30.00  50.00
emily_stu     30.00  50.00
joe           30.00  50.00
marcus_stu    30.00  50.00
nithin        30.00  50.00
priya_stu     30.00  50.00
prof_shannon  30.00  50.00
prof_turing   30.00  50.00
rahul_stu     30.00  50.00
staff         30.00  50.00
student       30.00  50.00
karthik        0.00  50.00
```

Purpose: Aggregates total customer lifetime spend vs current outstanding liabilities across all patrons.

17. Catalog Titles with Zero Hold Requests in Queue

```
SELECT Book_ID, Title, ISBN
FROM Books
WHERE Book_ID NOT IN (
    SELECT DISTINCT Book_ID FROM Loan_Requests WHERE Book_ID IS NOT NULL
)
ORDER BY Book_ID;
```

Query Result:

```
10  Database System Concepts (7th Ed)          978-0078022159
11  Modern Operating Systems (4th Ed)          978-0133591620
12  Computer Networks (5th Ed)                 978-0132126953
13  Domain-Driven Design: Tackling Complexity  978-0321125217
15  The Feynman Lectures on Physics            978-0465023820
```

Purpose: Uses a NOT IN subquery to identify titles with zero queue requests.

18. Chronological Security and Circulation Event Feed

```
SELECT Log_ID, Action_Type, Target_User, Performed_By,
       TO_CHAR(Log_Date, 'YYYY-MM-DD HH24:MI') AS Log_Time, Details
FROM Audit_Logs
ORDER BY Log_ID DESC
FETCH FIRST 6 ROWS ONLY;
```

Query Result:

```
19  Fine Waived            staff      admin      2026-10-04 16:30  Administrative waiver granted
for research staff loan.
18  Inventory Restocked    N/A        librarian  2026-10-02 11:15  Added new physical copies of
Computer Networks.
17  User Status Changed    rahul_stu  admin      2026-10-01 09:20  Patron account suspended due
to outstanding penalties.
16  Book Lost Reported     rahul_stu  librarian  2026-09-30 14:10  Book marked as Lost;
replacement fine billed.
15  Book Handover          student    librarian  2026-09-29 17:45  Handed over physical copy
from collection desk.
14  Loan Request Approved  student    librarian  2026-09-28 10:05  Approved loan request for
CLRS.
```

Purpose: Extracts timestamped administrative actions and system security events ordered chronologically.

8. DESCRIPTION

The architecture of the Library Management System is engineered around several core database design principles: strict separation of master data from transactional workflows, systematic normalization up to Boyce-Codd Normal Form (BCNF), concurrency serialization through pessimistic locks, and robust integrity enforcement via database triggers.

8.1 Master Data vs. Transaction Data Separation

A critical strength of the design is the decoupling of immutable bibliographic catalog data from volatile physical inventory items and circulation events. BOOKS defines the canonical intellectual entity (Title, ISBN, Publication Year, Category, Publisher). Individual physical copies are represented in BOOK_COPIES, allowing the library to stock dozens of physical copies of 'Clean Code', each maintaining its own operational status (Available, Borrowed, Lost, Maintenance, Reserved) and condition rating (New, Good, Fair, Poor). Circulation transactions in LOANS reference Copy_ID rather than Book_ID, avoiding data duplication and preventing update anomalies.

8.2 Normalization Analysis (1NF to BCNF)

The relational schema is systematically normalized to eliminate redundancy:

- **First Normal Form (1NF):** All relation attributes contain only atomic, indivisible values. Repeating groups (such as multiple authors for a book or multiple copies) are decoupled into separate relational tables (Book_Authors and Book_Copies). Primary keys uniquely identify every row.
- **Second Normal Form (2NF):** The database satisfies 1NF and contains zero partial key dependencies. In composite-key bridge relations like BOOK_AUTHORS (Book_ID, Author_ID), non-key author metadata is stored in AUTHORS, ensuring all non-prime attributes are fully functionally dependent on the entire composite primary key.
- **Third Normal Form (3NF):** The database satisfies 2NF and eliminates all transitive functional dependencies. For example, in BOOKS, Publisher_ID determines publisher address and contact details; storing publisher contact information in BOOKS would introduce a transitive dependency (Book_ID → Publisher_ID → Contact_Email). Decoupling publishers into PUBLISHERS eliminates this anomaly.
- **Boyce-Codd Normal Form (BCNF):** For every non-trivial functional dependency X → Y across all 11 relations, the determinant X is a superkey. There are no overlapping candidate keys or dependencies where a non-prime attribute determines a prime attribute. As a result, the database achieves complete BCNF compliance.

8.3 Concurrency Control & ACID Guarantees

In shared institutional terminals and high-concurrency environments, multiple patrons may attempt to checkout or reserve the same physical copy simultaneously. The system deploys pessimistic row-level locking (SELECT ... FOR UPDATE) inside sp_IssueBook. When a transaction accesses a copy, Oracle places an exclusive row-level lock on that Book_Copies record, forcing competing checkout threads to wait until the active transaction completes its validation and either commits or rolls back. This eliminates double-checkout race conditions.

8.4 Financial Ledger Integrity & Deletion Governance

FINES is kept separate from LOANS so that loans can transition seamlessly without locking financial ledger structures. The trigger trg_Secure_Fines_Deletion acts as an immutable ledger guard: physical DELETE operations on Fines are blocked at the database engine level unless authorized administrative

maintenance context is established. Fine amounts cannot be reduced unless Payment_Status is explicitly updated to 'Waived', ensuring tamper-proof audit trails for university financial compliance.

9. CONCLUSION

The Library Management System database was successfully architected, normalized, and implemented in Oracle Database 21c in strict accordance with the conceptual Entity-Relationship (ER) model and academic requirements for Digital Assignment 2. The 11 relational tables accurately represent categories, publishers, authors, books, physical shelf copies, role-based user accounts, circulation loans, overdue fines, digital hold requests, and administrative audit logs.

The implementation successfully unifies declarative SQL integrity constraints with advanced procedural PL/SQL components: encapsulated package architecture (LIBRARY_OPS_PKG), atomic book checkouts (sp_IssueBook), automated return and fine calculations (sp_ReturnBook), nightly batch synchronization (sp_AssessDailyFines), explicit cursor balances (fn_Get_Unpaid_Fines), ledger immutability triggers (trg_Secure_Fines_Deletion), real-time security auditors (trg_Audit_User_Suspension), and structured exception handling. The database guarantees high operational reliability, robust ACID compliance, and zero data redundancy, providing an exemplary foundation for modern university library management.